

Architecture and Design Overview

Agentic Software Engineering System — URL Shortener

Status: Proposed

Version: 1.0

Purpose: Architecture and design overview for the Agentic URL Shortener Assessment

1. Executive Summary

This solution consists of two cooperating systems:

1. **URL Shortener Application** — provides URL creation, redirection, basic analytics, validation, expiration handling, and persistent storage.
2. **Agentic Engineering Orchestrator** — coordinates AI-assisted engineering activities across requirements analysis, architecture, implementation, testing, validation, documentation, and release readiness.

The URL Shortener is intentionally implemented as a modular application rather than decomposed into independently deployed microservices. The assessment does not establish a requirement for distributed deployment, and introducing unnecessary infrastructure would increase operational complexity without materially improving the solution. Logical boundaries are maintained so components can be separated later if actual scale, deployment, or organizational requirements justify doing so.

The Agentic Engineering Orchestrator is the primary architectural differentiator. It uses an explicit dependency graph and stateful workflow to coordinate specialized agents while enforcing dependencies, governance, validation, approval gates, bounded retries, recovery, and safe-stop behavior.

The fundamental design principle is:

**Agents perform bounded engineering work; the orchestrator controls execution;
humans retain ownership of consequential decisions and final release approval.**

2. Architectural Goals

The architecture is designed to demonstrate the following capabilities.

Engineering Lifecycle

- Requirement interpretation and clarification
- Task decomposition

- Architecture and design analysis
- AI-assisted implementation
- Automated testing
- Security and quality validation
- Documentation generation
- Release-readiness assessment

Agentic Orchestration

- Explicit dependency graph
- Stateful workflow execution
- Sequential and parallel execution
- Synchronization
- Cross-stage context preservation
- Decision lineage
- Human approval checkpoints
- Bounded retries
- Fallback
- Rollback
- Safe-stop controls
- Policy guardrails
- Auditability
- Reliability metrics
- Dynamic replanning

URL Shortener

- URL shortening
- Redirect functionality
- Basic analytics
- Validation
- Persistence
- Expiration handling
- Automated testing
- Appropriate error handling

3. Non-Goals

The prototype intentionally does not attempt to implement:

- An Internet-scale URL-shortening platform
- A general-purpose autonomous software engineering platform
- Unrestricted AI-generated code execution
- Fully autonomous production deployment
- Multi-region infrastructure
- Enterprise identity management

- Comprehensive analytics or user profiling
- Collection of unnecessary personal information

These capabilities may be appropriate for a larger production system but are outside the scope of this assessment.

4. Architectural Principles

4.1 Controlled Autonomy

Agents are workers within a governed workflow. They do not independently control the overall engineering process.

The orchestrator determines:

- Which agent may execute
- When an agent may execute
- What context it receives
- What artifacts it may modify
- Which policies apply
- Whether human approval is required
- Whether a failure may be retried
- How the workflow recovers from failure

4.2 Human Accountability

Humans retain responsibility for consequential engineering decisions.

Human approval is required for actions such as:

- Major architectural changes
- Destructive database changes
- Security-sensitive changes
- Changes outside approved scope
- Production deployment
- Final release approval

4.3 Explicit State

Workflow state is represented explicitly rather than relying solely on an agent's conversational context.

This supports:

- Recovery

- Auditing
- Resumption
- Replanning
- Traceability
- Investigation

4.4 Validation Over Trust

AI-generated output is treated as proposed engineering work.

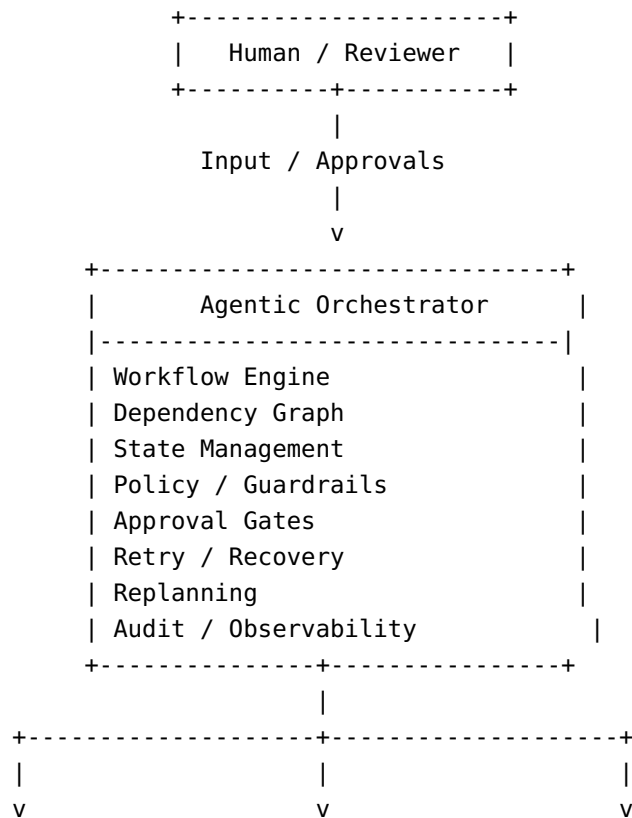
Code, architecture, tests, and documentation must pass appropriate validation before being accepted as completed workflow artifacts.

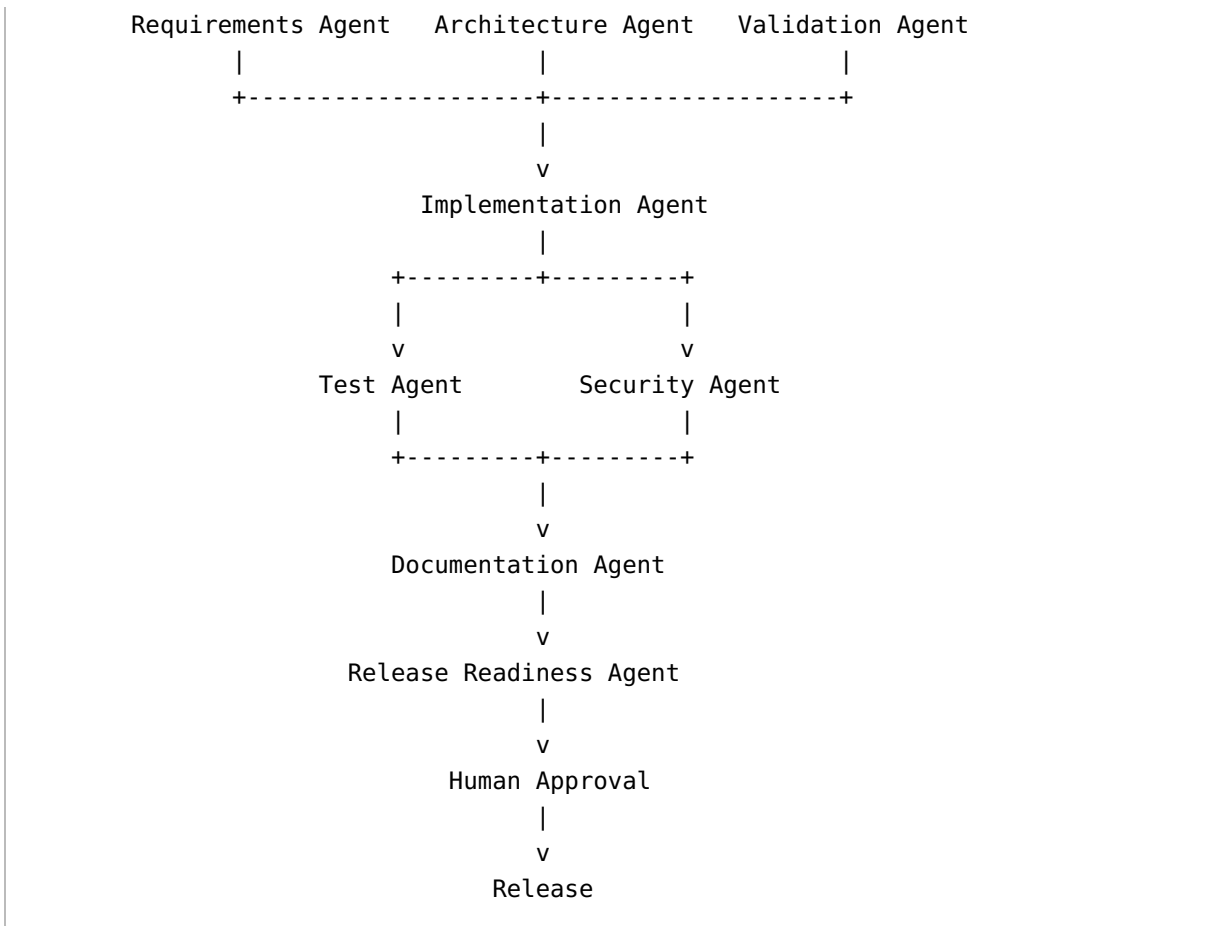
4.5 Justified Complexity

Architectural complexity should be driven by requirements rather than by the availability of technology.

The solution should prefer the simplest architecture that adequately demonstrates the required capabilities.

5. High-Level Architecture





The key architectural separation is between **orchestration** and **agent execution**.

Agents perform specialized engineering activities. The orchestrator controls the lifecycle in which those activities occur.

6. Agentic Orchestrator

6.1 Responsibilities

The orchestrator is responsible for:

- Loading workflow definitions
- Maintaining workflow state
- Resolving dependencies
- Determining runnable tasks
- Scheduling sequential and parallel work
- Synchronizing dependent branches
- Invoking agents

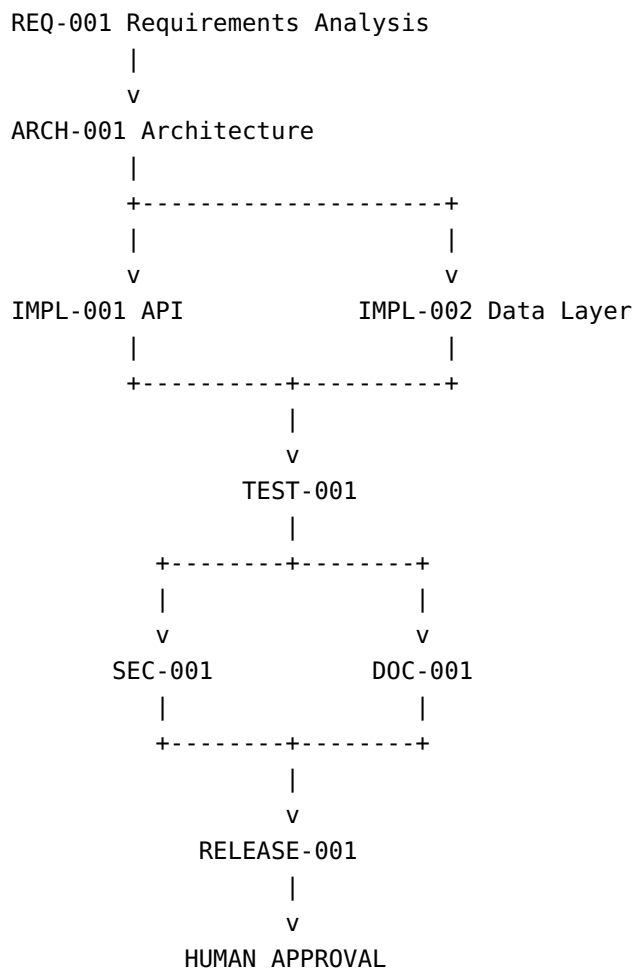
- Passing approved context between stages
- Enforcing policy
- Enforcing approval gates
- Managing retries
- Handling failures
- Triggering rollback
- Triggering replanning
- Safely stopping execution
- Recording execution history and metrics

The orchestrator should remain independent of URL-shortener-specific business logic.

7. Dependency Graph

Engineering work is represented as an explicit directed dependency graph.

A simplified workflow might be:



|
v
DONE

Each workflow node should contain, at minimum:

- Node ID
- Node type
- Dependencies
- Current status
- Input/context references
- Expected output
- Attempt count
- Validation status
- Artifact references
- Decision references

This makes the workflow an executable dependency graph rather than a fixed sequence of prompts.

8. Workflow State

A conceptual workflow state model is:

```
Created
|
v
Planning
|
v
Awaiting Approval
|
v
Executing
|
+-----> Blocked
|
+-----> Failed
|
v
Validating
|
+-----> Replanning
|           |
|           v
```



The implementation should distinguish at least:

- Runnable
- Running
- Completed
- Failed
- Blocked
- Awaiting approval
- Invalidated
- Replanning
- Safe-stopped

9. Agent Responsibilities

9.1 Requirements Agent

The Requirements Agent:

- Interprets incoming requirements
- Normalizes requirements
- Identifies ambiguity
- Produces acceptance criteria
- Identifies assumptions
- Identifies open questions
- Proposes task decomposition

It must not silently invent consequential requirements.

If clarification is necessary, the workflow transitions to a controlled waiting or safe-stop state.

9.2 Architecture Agent

The Architecture Agent:

- Analyzes requirements
- Proposes architecture
- Identifies affected components
- Identifies data flows
- Evaluates alternatives
- Identifies risks
- Produces architecture decisions

Architecture output should include rationale and trade-offs.

9.3 Implementation Agent

The Implementation Agent:

- Implements approved tasks
- Modifies source code within the approved scope
- Adds or updates tests
- Reports assumptions and blockers
- Produces a change summary

It should not independently expand the scope of an approved task.

9.4 Test Agent

The Test Agent:

- Generates or modifies tests
 - Executes automated tests
 - Analyzes failures
 - Identifies regression risk
 - Reports validation results
-

9.5 Security and Validation Agent

The Security/Validation Agent:

- Reviews security-sensitive changes
- Evaluates input validation
- Reviews configuration changes

- Identifies obvious security risks
 - Checks applicable policies
 - Performs validation against defined acceptance criteria
-

9.6 Documentation Agent

The Documentation Agent:

- Updates API documentation
 - Updates architecture documentation
 - Maintains setup/run instructions
 - Records relevant engineering decisions
 - Documents scenario execution and results
-

9.7 Release Readiness Agent

The Release Readiness Agent verifies:

- Build status
- Test status
- Required documentation
- Known risks
- Configuration requirements
- Validation results

It produces a release-readiness recommendation.

It does **not** independently authorize release.

10. Context and Artifact Management

Agents should receive structured context rather than an uncontrolled conversational transcript.

Typical context includes:

```
Workflow
Requirement
Acceptance Criteria
Current Task
Upstream Artifacts
Relevant Decisions
```

Repository State
Applicable Policies
Previous Validation Results
Known Risks

Engineering outputs should be represented as traceable artifacts.

Examples include:

- Normalized requirements
- Acceptance criteria
- Task plans
- Architecture proposals
- Architecture decisions
- Risk assessments
- Source-code changes
- Test results
- Security assessments
- Documentation
- Release-readiness reports

Artifacts should retain references to the workflow, task, agent, revision, dependencies, and validation state where practical.

11. Gates and Governance

11.1 Requirements Gate

Verify:

- Requirements are sufficiently understood
- Ambiguities have been identified
- Acceptance criteria exist
- Required clarification has been obtained

11.2 Architecture Gate

Verify:

- Architecture is documented
- Key decisions are documented
- Risks are identified
- Alternatives and trade-offs are recorded
- Required human approval has been obtained

11.3 Implementation Gate

Verify:

- Required implementation artifacts exist
- Changes remain within approved scope
- Basic validation succeeds

11.4 Test Gate

Verify:

- Required tests execute
- Required validation passes
- Known failures have been explicitly dispositioned

11.5 Release Gate

Verify:

- Build succeeds
 - Tests pass
 - Security/validation checks are complete
 - Required documentation exists
 - Known risks are documented
 - No critical unresolved findings remain
 - Required human approval is recorded
-

12. Controlled Autonomy Model

Actions are classified according to their potential impact.

Generally Autonomous

Examples:

- Requirement analysis
- Codebase analysis
- Task decomposition
- Documentation generation
- Test generation
- Technical analysis

Validation Required

Examples:

- Source-code modifications
- Database modifications
- API contract modifications
- Configuration changes
- Deployment configuration changes

Human Approval Required

Examples:

- Major architecture changes
- Destructive schema changes
- Security-sensitive changes
- Out-of-scope changes
- Production deployment
- Final release

The orchestrator must enforce these boundaries.

13. Failure and Recovery

13.1 Bounded Retry

Retryable operations have a maximum number of attempts.

```
Attempt
|
Failure
|
Retryable?
|
Yes
|
Attempts Remaining?
|
Yes
|
Retry
```

After the retry limit is reached, the workflow enters a controlled failure state.

Retry decisions must be recorded.

13.2 Fallback

An approved fallback may be used when:

- An agent provider is unavailable
- A preferred execution mechanism fails
- An alternative approved strategy exists

Fallback use must be observable in the audit history.

13.3 Rollback

Rollback returns affected workflow or artifact state to a known safe state.

The prototype does not need distributed transaction semantics.

The objective is to demonstrate that the orchestration system can recover from an unsuccessful consequential operation in a controlled manner.

13.4 Safe Stop

The workflow must safely stop when:

- Human approval is required
- Requirements remain unresolved
- A policy violation is detected
- Retry limits are exceeded
- An unsafe action is proposed
- Required dependencies cannot be satisfied

State must be preserved so the workflow can be investigated and, where appropriate, resumed.

14. Dynamic Replanning

New information may invalidate an existing plan.

Example:

```
Approved Architecture
  |
  v
Implementation
  |
  v
Technical Constraint Discovered
  |
  v
Architecture Impact Analysis
  |
  v
Affected Nodes Identified
  |
  v
Invalidate Stale Work
  |
  v
Revised Architecture
  |
  v
Approval Gate
  |
  v
Resume Execution
```

Only affected downstream work should be invalidated where practical.

Superseded decisions remain part of the historical decision lineage.

15. Decision Lineage

The system should maintain a traceable relationship between engineering artifacts.

```
Requirement
  |
  v
Architecture Decision
  |
  v
Implementation Task
  |
  v
```

```
Code Change
  |
  v
Test
  |
  v
Release Decision
```

A reviewer should be able to determine:

- Why a change was made
- Which requirement caused it
- Which architecture decision influenced it
- Which agent performed the work
- What validation occurred
- Whether human approval was required
- Whether the decision was later superseded

16. Audit and Observability

The orchestrator should produce an inspectable execution history.

A conceptual event might be:

```
{
  "workflowId": "workflow-001",
  "nodeId": "ARCH-001",
  "agent": "architecture-agent",
  "event": "approval-required",
  "timestamp": "2026-09-16T12:00:00Z",
  "reason": "Persistence architecture changed",
  "status": "waiting-for-human"
}
```

Useful event types include:

- workflow-created
- node-started
- node-completed
- node-failed
- retry-requested
- retry-executed
- approval-required

- approval-granted
- approval-rejected
- policy-violation
- artifact-created
- artifact-invalidated
- replan-started
- rollback-started
- rollback-completed
- safe-stop
- workflow-completed

17. Reliability Metrics

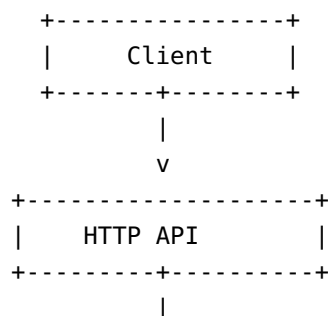
The system should capture enough information to report:

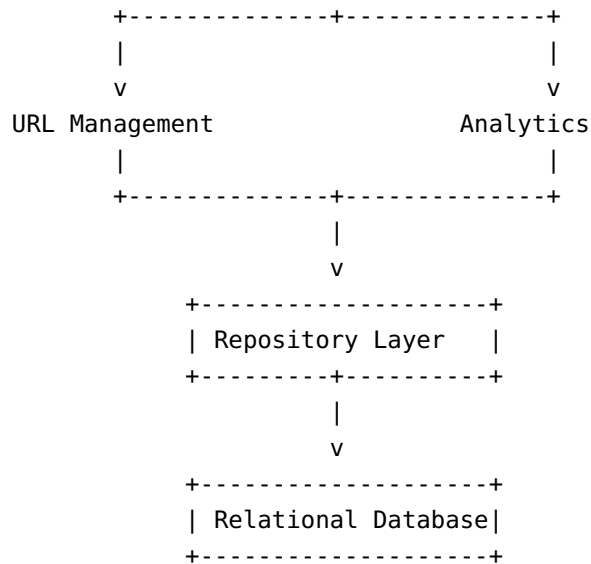
- Workflow success rate
- Agent/task success rate
- Retry frequency
- Rollback frequency
- Failure frequency
- End-to-end workflow latency
- Stage latency
- Mean time to recovery (MTTR), where meaningful
- Number of replans
- Number of human approval pauses

For this prototype, execution reports are sufficient. A dedicated enterprise telemetry platform is not required.

18. URL Shortener Architecture

18.1 Logical Components





The logical boundaries do not necessarily imply separate deployable services.

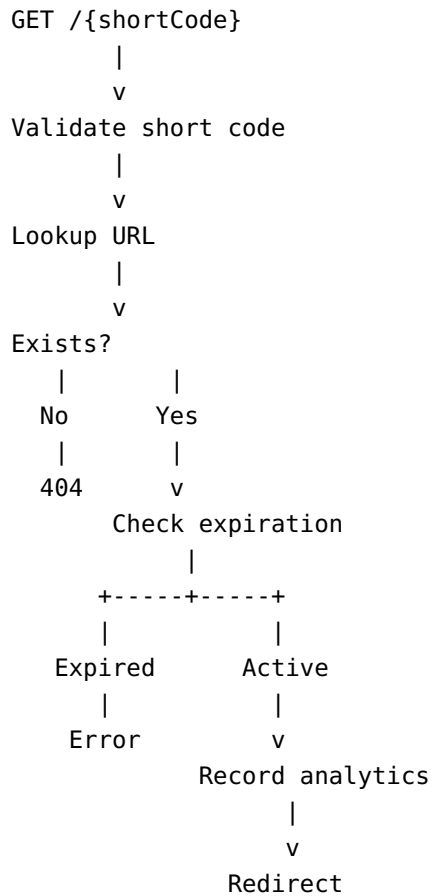
19. URL Creation Flow

```
POST /api/urls
  |
  v
Validate Request
  |
  v
Generate Short Code
  |
  v
Persist Mapping
  |
  v
Return Short URL
```

Validation should consider:

- Required fields
- URL format
- Maximum length
- Supported schemes
- Expiration constraints
- Custom alias rules, if supported

20. Redirect Flow



The redirect path should remain lightweight.

Analytics should not unnecessarily prevent successful redirection if the architecture can safely decouple non-critical analytics processing.

21. Analytics

The initial analytics implementation should remain deliberately small.

Potential metrics:

- Total redirects
- Redirects per shortened URL
- Redirect timestamps

Personal data should not be collected unless explicitly required.

If future requirements create significant throughput or latency requirements, an event-driven analytics path could be introduced.

For the prototype, synchronous persistence is acceptable if its trade-offs are documented.

22. Data Model

A conceptual model is:

```
ShortUrl
-----
Id
ShortCode
OriginalUrl
CreatedAt
ExpiresAt
IsActive
```

```
RedirectEvent
-----
Id
ShortUrlId
OccurredAt
```

The database should enforce uniqueness on `ShortCode` .

If generated codes collide:

1. Detect the persistence conflict.
 2. Generate another code.
 3. Retry within a bounded limit.
 4. Return an appropriate failure if the limit is exceeded.
-

23. API Design

An initial API surface may be:

```
POST /api/urls
GET /api/urls/{shortCode}
GET /api/urls/{shortCode}/analytics
GET /{shortCode}
```

The final API should be established during requirements analysis.

API documentation should define:

- Request schemas
- Response schemas
- HTTP status codes
- Validation errors
- Not-found behavior
- Expiration behavior
- Authentication requirements, if applicable

24. Security Design

The system should evaluate:

- URL and input validation
- Safe URL schemes
- SQL injection prevention
- Secret/configuration handling
- Error information disclosure
- Abuse/rate-limiting considerations
- Open-redirect implications
- Analytics privacy

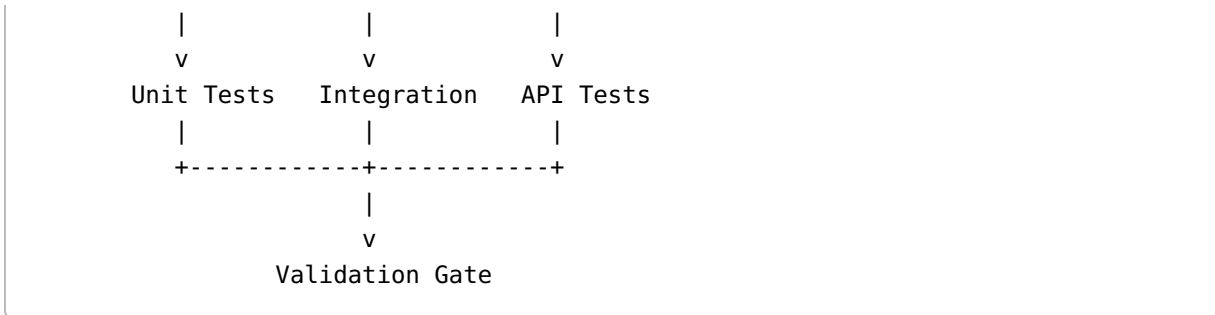
Internal exception details must not be exposed through public API responses.

Security findings and remaining limitations must be documented.

25. Testing Architecture

The application should use multiple test levels.

```
Automated Tests
|
+-----+-----+
```



The orchestration engine requires its own tests covering:

- Dependency resolution
- Parallel execution
- Synchronization
- Approval gates
- Retry limits
- Safe stop
- Rollback
- Replanning
- Context propagation
- Policy enforcement
- State transitions

The orchestrator must not rely exclusively on the URL Shortener tests for validation.

26. Required Demonstration Scenarios

26.1 Greenfield

Requirement:

Build a URL shortener supporting URL creation, redirect, and basic analytics.

Demonstrates:

```

Requirements
  ↓
Decomposition
  ↓
Architecture
  ↓
Approval
  ↓
Implementation
  
```

```
↓
Testing
↓
Validation
↓
Documentation
↓
Release Readiness
```

26.2 Brownfield

Example requirement:

Add URL expiration without breaking existing behavior.

Demonstrates:

```
Requirement
↓
Codebase Analysis
↓
Impact Analysis
↓
Data/API/Test Impact
↓
Implementation
↓
Regression Testing
↓
Validation
```

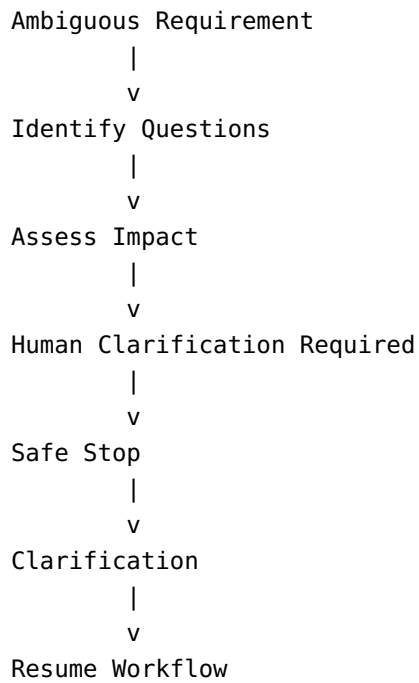
The purpose is to demonstrate reasoning about an existing system rather than treating every request as greenfield development.

26.3 Ambiguous Requirement

Example:

We need better analytics for shortened URLs.

Expected behavior:



Potential clarification questions include:

- What metrics are required?
- What constitutes a click?
- Per URL or globally?
- Are timestamps required?
- Are unique visitors required?
- Is IP address collection permitted?
- Is geographic information required?
- What is the retention period?
- Who may access analytics?
- Is reporting real-time or eventually consistent?
- What privacy requirements apply?

The system should not silently invent consequential requirements.

27. Architecture Decision Records

Significant architectural decisions should be documented as ADRs.

Recommended decisions include:

- Modular monolith vs. microservices
- Database selection

- Short-code generation
- Analytics architecture
- Agent boundaries
- Workflow state model
- Dependency graph representation
- Approval model
- Retry policy
- Rollback strategy
- Replanning strategy
- Security policies

Recommended ADR format:

```
# ADR-XXX: <Decision>

## Status

Proposed / Accepted / Superseded

## Context

What problem requires a decision?

## Decision

What was selected?

## Alternatives

What alternatives were considered?

## Rationale

Why was this selected?

## Trade-offs

What benefits and costs result?

## Consequences

What follows from this decision?
```

28. Deployment and Release Readiness

The repository should be runnable from a clean checkout.

Required documentation includes:

- Prerequisites
- Build instructions
- Database setup
- Configuration
- Run instructions
- Test instructions
- Scenario execution

The release-readiness process should follow:

```
Clean Checkout
  |
  v
Build
  |
  v
Automated Tests
  |
  v
Security / Validation
  |
  v
Documentation
  |
  v
Risk Review
  |
  v
Human Release Approval
```

29. Proposed Repository Structure

```
/
├── README.md
├──
└── docs/
```

```

|   |   | architecture.md
|   |   | orchestration.md
|   |   | engineering-decisions.md
|   |   | risk-register.md
|   |   | scenarios.md
|   |   | test-strategy.md
|   |   | release-readiness.md
|   |
|   |--- src/
|   |   |--- UrlShortener/
|   |   |--- Orchestrator/
|   |   |--- Agents/
|   |   |--- Shared/
|   |
|   |--- tests/
|   |   |--- UrlShortener.UnitTests/
|   |   |--- UrlShortener.IntegrationTests/
|   |   |--- Orchestrator.Tests/
|   |
|   |--- scenarios/
|   |   |--- greenfield/
|   |   |--- brownfield/
|   |   |--- ambiguous/
|   |
|   |--- artifacts/
|   |   |--- execution traces and reports

```

The final structure may change as implementation decisions are made.

30. Key Architectural Trade-Offs

Modular Monolith vs. Microservices

Proposed decision: Modular monolith.

Rationale: The assessment does not establish a requirement for independently deployed services. A modular monolith minimizes operational complexity while preserving logical boundaries.

Future option: Extract services if scale, deployment independence, or organizational boundaries create a concrete justification.

Synchronous vs. Asynchronous Analytics

Proposed decision: Begin with the simplest implementation satisfying the requirements.

Trade-off: Synchronous persistence is simpler but couples redirect processing to analytics persistence.

Future option: Introduce event-driven analytics if throughput or latency requirements justify the additional complexity.

Generic vs. Assessment-Focused Orchestrator

Proposed decision: Build an assessment-focused orchestration engine.

Rationale: A fully generic AI engineering platform would substantially increase complexity without proportionally improving the assessment demonstration.

Autonomous vs. Controlled Execution

Proposed decision: Controlled autonomy.

Rationale: Consequential engineering decisions require validation, traceability, and human ownership.

31. Key Risks and Mitigations

Risk	Mitigation
Agent produces incorrect code	Automated tests and validation gates
Agent invents requirements	Ambiguity detection and clarification gate
Agent makes excessive changes	Scope and policy guardrails
Infinite retry loops	Bounded retry limits
Stale downstream work	Dependency-aware invalidation
Destructive operations	Human approval policy
Hidden workflow state	Persistent and inspectable state
Difficult reviewer setup	Simple local execution
Over-engineering	Explicit non-goals and architecture constraints
Unnecessary analytics collection	Data minimization

Risk	Mitigation
Documentation drift	Documentation validation during release readiness

32. Architecture Validation Criteria

The implementation should be considered architecturally complete when:

Orchestration

- ☐ Dependency graph is explicit
- ☐ Workflow state is explicit
- ☐ Sequential execution works
- ☐ Parallel execution works
- ☐ Synchronization works
- ☐ Context is preserved
- ☐ Decision lineage exists
- ☐ Approval gates exist
- ☐ Retry is bounded
- ☐ Fallback is demonstrated where appropriate
- ☐ Rollback is demonstrated
- ☐ Safe-stop is demonstrated
- ☐ Replanning is demonstrated
- ☐ Policies are enforced
- ☐ Audit history is produced
- ☐ Reliability metrics are available

URL Shortener

- ☐ URL creation works
- ☐ Redirect works
- ☐ Analytics work
- ☐ Validation exists
- ☐ Expiration works
- ☐ Persistence works
- ☐ Appropriate error handling exists
- ☐ Security considerations are addressed

Quality

- ☐ Unit tests pass
- ☐ Integration tests pass
- ☐ API tests pass
- ☐ Orchestrator tests pass
- ☐ Documentation matches implementation
- ☐ Clean checkout has been verified

- [] No secrets are committed
- [] Known limitations are documented

33. Final Architectural Position

The architecture deliberately treats the **agentic orchestration layer as the primary engineering subject** and the URL Shortener as its reference workload.

The resulting system demonstrates the complete engineering lifecycle:

```
Requirement
  ↓
Understanding
  ↓
Decomposition
  ↓
Architecture
  ↓
Controlled Agent Execution
  ↓
Validation
  ↓
Human Governance
  ↓
Release
```

It also explicitly handles non-happy-path execution:

```
Failure
  → Retry / Fallback / Rollback

Ambiguity
  → Human Clarification

Policy Violation
  → Safe Stop

New Information
  → Replan

Changed Architecture
  → Invalidate Affected Work
```

The design therefore combines agentic execution with conventional software engineering fundamentals:

- Explicit requirements
- Modular architecture
- Testability
- Automated validation
- Change control
- Security
- Observability
- Traceability
- Risk management
- Human accountability

The goal is not to demonstrate that an AI agent can generate code autonomously. The goal is to demonstrate that AI agents can perform useful engineering work within a system that preserves **engineering judgment, controlled autonomy, validation, governance, and accountability.**